SKILL WEEK-12

Task1 : To execute large pipelines, launch multiple jobs, measure system stability, monitor resource usage, detect failures, and analyze performance.

Aim : To implement a C program that creates multiple child processes, executes pipeline commands, monitors their execution, detects failures, and measures the overall execution time.

Step 1 - Create a C file:

nano pipeline_monitor.c

```
  GNU nano 7.2                    pipeline_monitor.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

int main()
{
    int p1[2], p2[2];
    pid_t p[3];
    int status;
    clock_t start, end;

    start = clock();

    pipe(p1);
    pipe(p2);

    p[0] = fork();

    if (p[0] == 0)
    {
        dup2(p1[1], STDOUT_FILENO);
        close(p1[0]);
        close(p1[1]);
        close(p2[0]);
        close(p2[1]);

        execlp("ls", "ls", NULL);
        exit(1);
    }

    p[1] = fork();
```

```c
    if (p[1] == 0)
    {
        dup2(p1[0], STDIN_FILENO);
        dup2(p2[1], STDOUT_FILENO);
        close(p1[0]);
        close(p1[1]);
        close(p2[0]);
        close(p2[1]);

        execlp("grep", "grep", ".c", NULL);
        exit(1);
    }

    p[2] = fork();

    if (p[2] == 0)
    {
        dup2(p2[0], STDIN_FILENO);
        close(p1[0]);
        close(p1[1]);
        close(p2[0]);
        close(p2[1]);

        execlp("wc", "wc", "-l", NULL);
        exit(1);
    }

    close(p1[0]);
    close(p1[1]);
    close(p2[0]);
    close(p2[1]);
```

```c
    for (int i = 0; i < 3; i++)
    {
        waitpid(p[i], &status, 0);

        if (WIFEXITED(status) && WEXITSTATUS(status) == 0)
            printf("Child %d completed successfully\n", i + 1);
        else
            printf("Child %d failed\n", i + 1);
    }

    end = clock();

    printf("Pipeline completed\n");
    printf("Execution time: %.6f seconds\n",
           (double)(end - start) / CLOCKS_PER_SEC);

    return 0;
}
```

Step 2: Compile and run the program

```
alurusaranya@LAPTOP-0PA61QEN:~$ cd ~/OSSP
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ mkdir Week12
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ cd Week12
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12$ pwd
/home/alurusaranya/OSSP/Week12
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12$ nano pipeline_monitor.c
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12$ gcc pipeline_monitor.c -o pipeline_monitor
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12$ ./pipeline_monitor
Child 1 completed successfully
1
Child 2 completed successfully
Child 3 completed successfully
Pipeline completed
Execution time: 0.001126 seconds
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12$
```

Observation

A C program was used to create three child processes and execute the pipeline ls → grep .c → wc -l. Pipes were used for communication and waitpid() was used to monitor the processes. The program also detected failures and measured execution time.

Result

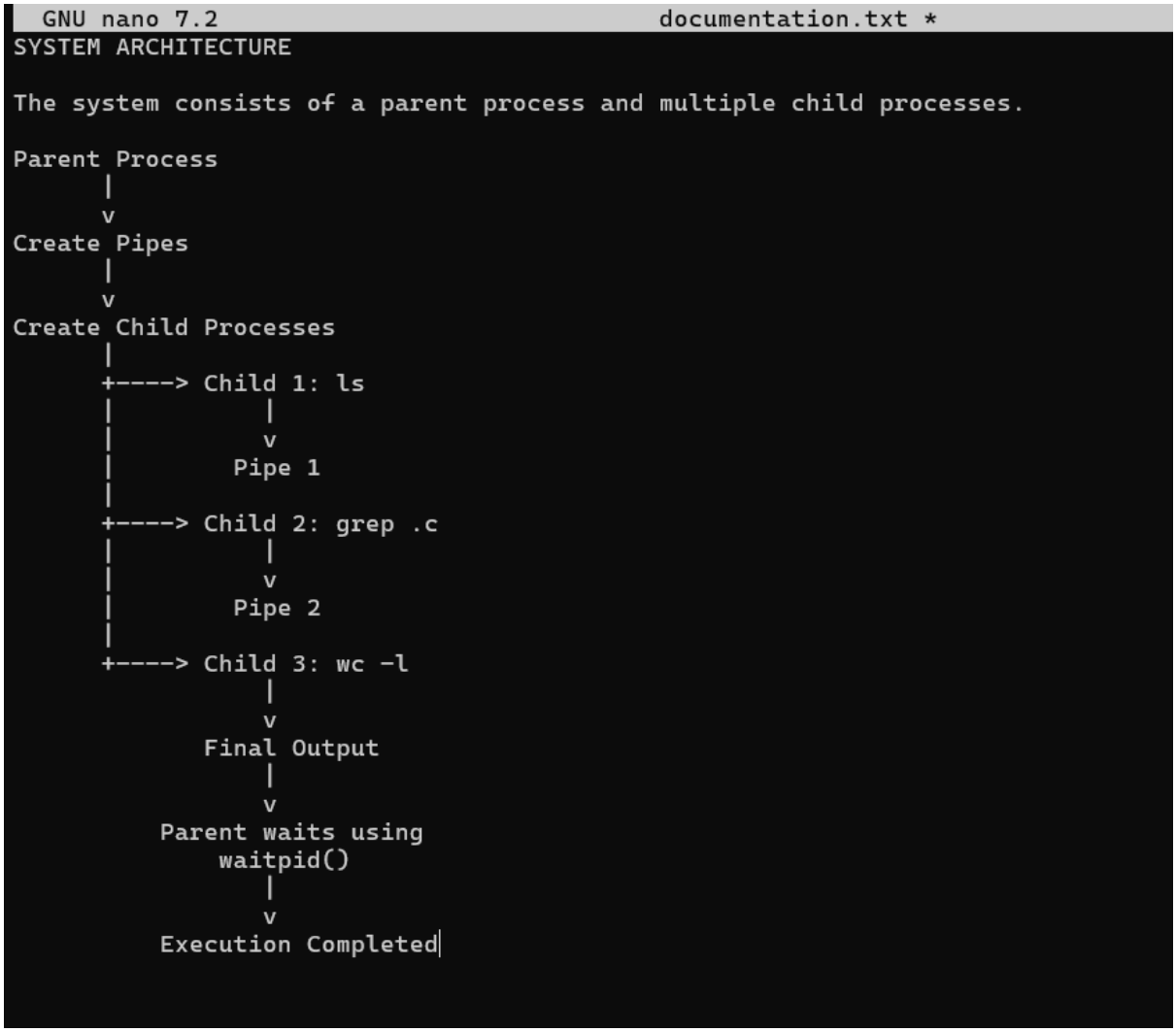
The pipeline was executed successfully. All child processes completed successfully, failure detection was verified, and the overall execution time was measured.

Task2 : To document the system architecture, create flow diagrams, explain execution paths, describe system calls, record design decisions, and review the project documentation.

Aim : To document the architecture and execution flow of the system, identify the system calls used, record important design decisions, and maintain clear and organized project documentation.

Step1: Create the documentation file:

nano documentation.txt



Step 2: Add Design Decisions

```
  GNU nano 7.2                                    documentation.txt *
SYSTEM ARCHITECTURE

The system consists of a parent process and multiple child processes.

Parent Process
      |
      v
Create Pipes
      |
      v
Create Child Processes
      |
      +----> Child 1: ls
      |          |
      |          v
      |        Pipe 1
      |
      +----> Child 2: grep .c
      |          |
      |          v
      |        Pipe 2
      |
      +----> Child 3: wc -l
                 |
                 v
            Final Output
                 |
                 v
          Parent waits using
              waitpid()
                 |
                 v
          Execution Completed


DESIGN DECISIONS

1. Pipes are used for communication between processes.
2. fork() is used to create multiple child processes.
3. dup2() is used to connect the input and output of each pipeline stage.
4. execlp() is used to execute Linux commands.
5. waitpid() is used to monitor child processes and detect failures.
6. clock() is used to measure the overall execution time.
7. The parent process closes unused pipe ends to avoid resource problems.
```

Step 4: Add Project Documentation Review

```
DESIGN DECISIONS

1. Pipes are used for communication between processes.
2. fork() is used to create multiple child processes.
3. dup2() is used to connect the input and output of each pipeline stage.
4. execlp() is used to execute Linux commands.
5. waitpid() is used to monitor child processes and detect failures.
6. clock() is used to measure the overall execution time.
7. The parent process closes unused pipe ends to avoid resource problems.


DOCUMENTATION REVIEW

The architecture, execution flow, system calls, and design decisions
have been documented clearly. The documented flow was compared with
the implemented program and found to be consistent.

The system successfully creates child processes, connects them using
pipes, executes the pipeline commands, monitors their status, and
measures execution time.
```

Observation

The system architecture, execution flow, system calls, and design decisions were documented. The documented flow was reviewed and matched with the implemented pipeline program.

Result

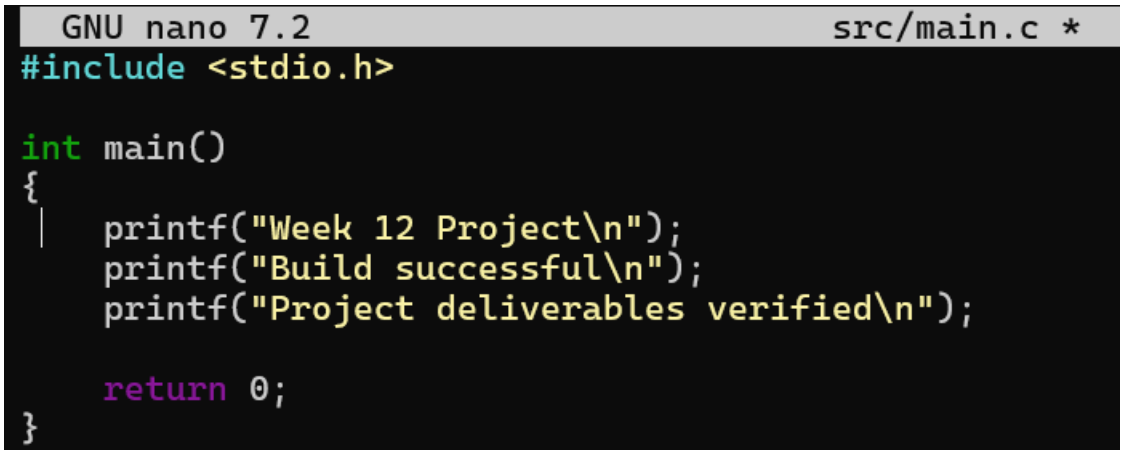
The system documentation was successfully prepared and reviewed, including the architecture, execution path, system calls, and important design decisions.

Task3 : To organize source code, improve build scripts, add a README file, verify repository structure, tag releases, and validate project deliverables.

Aim : To maintain a well-organized project repository with a reliable build process, clear documentation, proper version tags, and verified project deliverables.

Step 1: Create the Source Code

nano src/main.c



Step 2: Create the Makefile

Nano Makefile

```
  GNU nano 7.2                                  Makefile *
CC = gcc
CFLAGS = -Wall
TARGET = build/week12
SOURCE = src/main.c

all:
        $(CC) $(CFLAGS) $(SOURCE) -o $(TARGET)

run: all
        ./$(TARGET)

clean:
        rm -f $(TARGET)
```

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ mkdir Week12_Project
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ cd Week12_Project
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ mkdir src build
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ nano src/main.c
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ Step 3: Create the Makefile
Command 'Step' not found, did you mean:
  command 'step' from deb step (4:23.08.4-0ubuntu1)
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ nano Makefile
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ make
gcc -Wall src/main.c -o build/week12
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ make run
gcc -Wall src/main.c -o build/week12
./build/week12
Week 12 Project
Build successful
Project deliverables verified
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$
```

Step 3: Create the README File

nano README.md

```
  GNU nano 7.2                                  README.md *
# Week 12 Project

## Description
This project demonstrates process creation, pipeline execution,
process monitoring, and project organization.

## Build
```bash
make
```
```

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ nano README.md
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ make run
gcc -Wall src/main.c -o build/week12
./build/week12
Week 12 Project
Build successful
Project deliverables verified
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ make clean
rm -f build/week12
```

Step 5: Initialize Git and Create a Release Tag

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ pwd
/home/alurusaranya/OSSP/Week12_Project
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/alurusaranya/OSSP/Week12_Project/.git/
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ git add .
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ git commit -m "Add Week 12 project deliverab
les"
[master (root-commit) a74bfbf] Add Week 12 project deliverables
 3 files changed, 32 insertions(+)
 create mode 100644 Makefile
 create mode 100644 README.md
 create mode 100644 src/main.c
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ git tag -a v1.0 -m "Week 12 release"
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ git tag
v1.0
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ git status
On branch master
nothing to commit, working tree clean
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$
```

Step-6: Verifying files

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ find . -maxdepth 2 -type f
./README.md
./.git/COMMIT_EDITMSG
./.git/HEAD
./.git/config
./.git/description
./.git/index
./src/main.c
./Makefile
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ make clean
make
rm -f build/week12
gcc -Wall src/main.c -o build/week12
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ ./build/week12
Week 12 Project
Build successful
Project deliverables verified
```

Observation:
The project files were organized into source, build, Makefile, and README sections. The Makefile successfully compiled and executed the program, and the project structure was verified locally.

Result:
The Week 12 project was successfully organized, built, documented, and verified with all required deliverables.

Task4 : To create demo scenarios, validate project features, rehearse the presentation, demonstrate pipelines, demonstrate signal handling, and perform the final project review.

Aim : To verify the complete project functionality through practical demonstrations, test important features such as pipelines and signals, and prepare the project for final presentation and evaluation.

Step 1: Go t pipeline_monitor.c

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12_Project$ cd ~/OSSP/Week12
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12$ ls
documentation.txt  pipeline_monitor  pipeline_monitor.c
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12$ pipeline_monitor.c
pipeline_monitor.c: command not found
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12$ gcc pipeline_monitor.c -o pipeline_monitor
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12$ ./pipeline_monitor
2
Child 1 completed successfully
Child 2 completed successfully
Child 3 completed successfully
Pipeline completed
Execution time: 0.000422 seconds
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12$
```

Step 2: Demonstrate Signal Handling

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP/Week12$ cd ~/OSSP
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ find . -name "sigint_forward.c"
./sigint_forward.c
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ gcc sigint_forward.c -o sigint_forward
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./sigint_forward
Simple shell started.
Shell PID: 747

shell> Starting foreground job...
Foreground job PID: 748
Press Ctrl+C to interrupt the foreground job.
Foreground job started. PID = 748
Foreground job is running...
Foreground job is running...
^C
Shell received SIGINT.
No foreground job is running.
Foreground job is running...

Shell received SIGINT.
Forwarding SIGINT to foreground job (PID = 748)
Job state updated: TERMINATED
Shell is still running.

Shell exiting normally.

Shell received SIGINT.
No foreground job is running.
Foreground job is running...
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ Foreground job is running...
Foreground job is running...
Foreground job is running...
^C
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ Foreground job is running...
Foreground job is running...
```

Observation:

The project features were demonstrated and reviewed. Pipeline execution, signal handling, process monitoring, documentation, and build functionality were verified through practical demonstrations.

Result:

The complete project functionality was successfully verified, and the project was prepared for final presentation and evaluation.